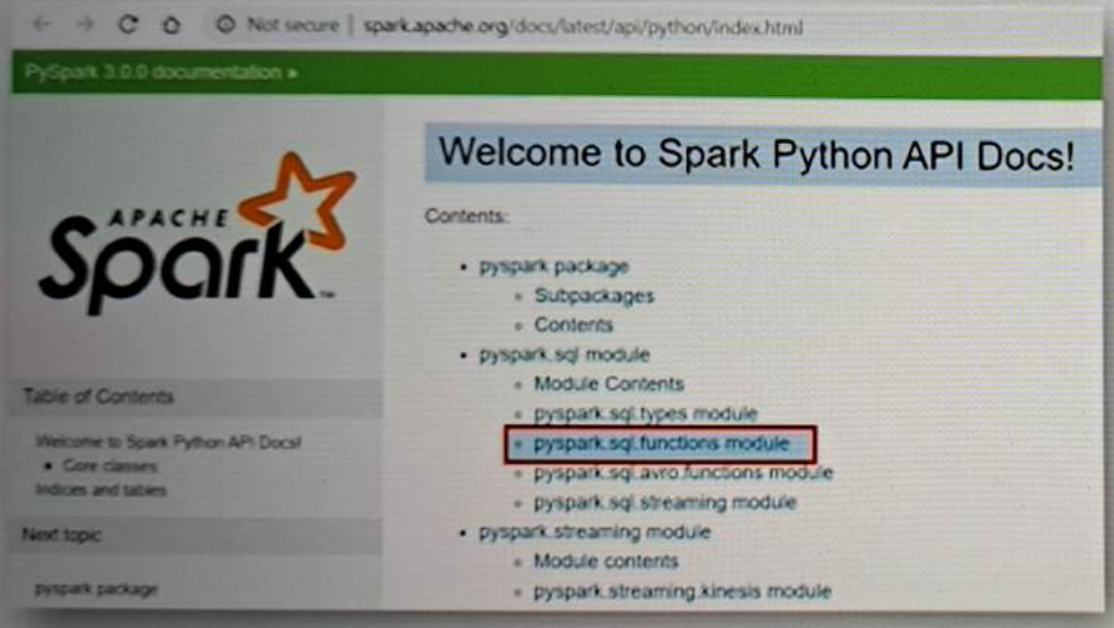
Aggregations

1. Simple Aggregations
2. Grouping Aggregations
3. Windowing Aggregations

LEARNING JOURNAL
www.learningjournal.guru

Spark Aggregations



Aggregating Functions

- avg()
- count()
- max()
- min()
- sum()

Window Functions

- lead()
- lag()
- rank()
- dense_rank()
- cume_dist()

LEARNING JOURNAL
www.learningjournal.guru

File Edit View Navigate Code Refactor Run Tools VCS Window Help 13-AggDemo - AggDemo.py - PyCharm

13-AggDemo / AggDemo.py

Project

- 13-AggDemo
  - app-logs
  - data
    - invoices.csv
  - lib
  - spark-warehouse
  - AggDemo.py
  - log4j.properties
- External Libraries
- Scratches and Consoles

AggDemo.py

```python
    spark = SparkSession \
        .builder \
        .appName("Agg Demo") \
        .master("local[2]") \
        .getOrCreate()

    logger = Log4j(spark)

    invoice_df = spark.read \
        .format("csv") \
        .option("header", "true") \
        .option("inferSchema", "true") \
        .load("data/invoices.csv")

    invoice_df.select(f.count("*").alias("Count *"),
                      f.sum("Quantity").alias("TotalQuantity"),
                      f.avg("UnitPrice").alias("AvgPrice"),
                      f.countDistinct("InvoiceNo").alias("CountDistinct")).show()
```

Run: AggDemo

```
|Count *|TotalQuantity|        AvgPrice|CountDistinct|
+-------+-------------+----------------+-------------+
| 541909|      5176450|4.611113626888512|        25900|

Process finished with exit code 0
```

Git TODO Run Terminal Python Console

PEP 8: W391 blank line at end of file

Event Log

25:5 CRLF UTF-8 4 spaces Python 3.7 (pyspark45) master

File Edit View Navigate Code Refactor Run Tools VCS Window Help 13-AggDemo - AggDemo.py - PyCharm

13-AggDemo | AggDemo.py

Project

- 13-AggDemo C:\Users\prashant\Documents\GitHub\Spark-Proj
  - app-logs
  - data
    - invoices.csv
  - lib
  - spark-warehouse
  - AggDemo.py
  - log4j.properties
- External Libraries
- Scratches and Consoles

AggDemo.py

```
        .format("csv") \
        .option("header", "true") \
        .option("inferSchema", "true") \
        .load("data/invoices.csv")

    invoice_df.select(f.count("*").alias("Count *"),
                      f.sum("Quantity").alias("TotalQuantity"),
                      f.avg("UnitPrice").alias("AvgPrice"),
                      f.countDistinct("InvoiceNo").alias("CountDistinct")).show()

    invoice_df.selectExpr(
        "count(1) as `count 1`",
        "count(StockCode) as `count field`",
        "sum(Quantity) as TotalQuantity",
        "avg(UnitPrice) as AvgPrice"
    ).show()
if __name__ == "__main__"
```

Run: AggDemo

```
+--------+-------------+-----------------+-------------+
|Count *|TotalQuantity|         AvgPrice|CountDistinct|
+--------+-------------+-----------------+-------------+
| 541909|      5176450|4.611113626088512|        25900|
+--------+-------------+-----------------+-------------+

+--------+-----------+-------------+-----------------+
|count 1|count field|TotalQuantity|         AvgPrice|
+--------+-----------+-------------+-----------------+
| 541909|     541908|      5176450|4.611113626882965|
+--------+-----------+-------------+-----------------+

Process finished with exit code 0
```

Git | TODO | Run | Terminal | Python Console | Event Log

Grouping Aggregations?

udemy

File Edit View Navigate Code Refactor Run Tools VCS Window Help 13-AggDemo - AggDemo.py - PyCharm

13-AggDemo | AggDemo.py

Project

- 13-AggDemo C:\Users\prashant\Documents\GitHub\Spark-Proj
  - app-logs
  - data
    - invoices.csv
  - lib
  - spark-warehouse
  - AggDemo.py
  - log4j.properties
- External Libraries
- Scratches and Consoles

AggDemo.py

```
        "count(StockCode) as `count field`",
        "sum(Quantity) as TotalQuantity",
        "avg(UnitPrice) as AvgPrice"
    ).show()
    invoice_df.createOrReplaceTempView("sales")
    summary_sql = spark.sql("""
    SELECT Country, InvoiceNo,
            sum(Quantity) as TotalQuantity,
            round(sum(Quantity * UnitPrice),2) as InvoiceValue
    FROM sales
    GROUP BY Country, InvoiceNo""")
    summary_sql.show()

if __name__ == "__main__"
```

Run: AggDemo

```
+--------+-----------+-------------+----------------+

+--------------+---------+-------------+------------+
|       Country|InvoiceNo|TotalQuantity|InvoiceValue|
+--------------+---------+-------------+------------+
|United Kingdom|   536446|          329|      440.89|
|United Kingdom|   536508|          216|      155.52|
|United Kingdom|   537018|           -3|         0.0|
|United Kingdom|   537401|          -24|         0.0|
|United Kingdom|   537811|           74|      268.86|
|United Kingdom|  C537824|           -2|       -14.9|
|United Kingdom|   538895|          370|      247.38|
|United Kingdom|   548453|          341|      302.45|
|United Kingdom|   541291|          217|      305.81|
|United Kingdom|   542551|           -1|         0.0|
|United Kingdom|   542576|           -1|         0.0|
```

9 Git | 6 TODO | 4 Run | Terminal | Python Console | Event Log

39:23 CRLF UTF-8 4 spaces Python 3.7 (pyspark45) master

```python
invoice_df.selectExpr(
    "count(1) as `count 1`",
    "count(StockCode) as `count field`",
    "sum(Quantity) as TotalQuantity",
    "avg(UnitPrice) as AvgPrice"
).show()
invoice_df.createOrReplaceTempView("sales")
summary_sql = spark.sql("""
SELECT Country, InvoiceNo,
        sum(Quantity) as TotalQuantity,
        round(sum(Quantity * UnitPrice),2) as InvoiceValue
FROM sales
GROUP BY Country, InvoiceNo""")
summary_sql.show()

summary_df = invoice_df \
    .groupBy("Country", "InvoiceNo") \
    .agg(f.sum("Quantity").alias("TotalQuantity"),
         f.round(f.sum(f.expr("Quantity * UnitPrice")),2).alias("InvoiceValue"),
         f.expr("round(sum(Quantity * UnitPrice),2) as InvoiceValueExpr"))
```

Aggregations

Country	WeekNumber	NumInvoices	TotalQuantity	InvoiceValue
EIRE	48	7	2822	3147.23
EIRE	49	5	1280	3284.1
EIRE	50	5	1184	2321.78
EIRE	51	5	95	276.84
France	48	4	1299	2808.16
France	49	9	2303	4527.01
France	50	6	529	537.32
France	51	5	847	1702.87

LEARNING JOURNAL
www.learningjournal.guru

```
        .load("data/invoices.csv")

    NumInvoices = f.countDistinct("InvoiceNo").alias("NumInvoices")
    TotalQuantity = f.sum("Quantity").alias("TotalQuantity")
    InvoiceValue = f.expr("round(sum(Quantity * UnitPrice),2) as InvoiceValue")

    exSummary_df = invoice_df \
        .withColumn("InvoiceDate", f.to_date(f.col("InvoiceDate"), "dd-MM-yyyy H.mm")) \
        .where("year(InvoiceDate) == 2010") \
        .withColumn("WeekNumber", f.weekofyear(f.col("InvoiceDate"))) \
        .groupBy("Country", "WeekNumber") \
        .agg(NumInvoices, TotalQuantity, InvoiceValue)
    exSummary_df\
        .write \
        .format("parquet") \
        .mode("overwrite") \
        .save("output")

if __name__ == "__main__"
```

File Edit View Navigate Code Refactor Run Tools VCS Window Help 14-GroupingDemo - GroupingDemo.py - PyCharm

14-GroupingDemo > GroupingDemo.py

Project

- 14-GroupingDemo C:\Users\prashant\Documents\
  - app-logs
  - data
  - lib
  - output
  - GroupingDemo.py
  - log4j.properties
- External Libraries
- Scratches and Consoles

GroupingDemo.py

```
23      InvoiceValue = f.expr("round(sum(Quantity * UnitPrice),2) as InvoiceValue")
24
25      exSummary_df = invoice_df \
26          .withColumn("InvoiceDate", f.to_date(f.col("InvoiceDate"), "dd-MM-yyyy H.mm")) \
27          .where("year(InvoiceDate) == 2010") \
28          .withColumn("WeekNumber", f.weekofyear(f.col("InvoiceDate"))) \
29          .groupBy("Country", "WeekNumber") \
30          .agg(NumInvoices, TotalQuantity, InvoiceValue)
31      exSummary_df.coalesce(1) \
32          .write \
33          .format("parquet") \
34          .mode("overwrite") \
```

Run: GroupingDemo

```
|       Country|WeekNumber|NumInvoices|TotalQuantity|InvoiceValue|
+--------------+----------+-----------+-------------+------------+
|     Australia|        48|          1|          107|      358.25|
|     Australia|        49|          1|          214|       258.9|
|     Australia|        50|          2|          133|      387.95|
|       Austria|        50|          2|            3|      257.04|
|       Bahrain|        51|          1|           54|      205.74|
|       Belgium|        48|          1|          528|       346.1|
|       Belgium|        50|          2|          285|      625.16|
|       Belgium|        51|          2|          942|      838.65|
|Channel Islands|       49|          1|           80|      363.53|
|        Cyprus|        50|          1|          917|     1590.82|
|       Denmark|        49|          1|          454|      1281.5|
|          EIRE|        48|          7|         2822|     3147.23|
|          EIRE|        49|          5|         1280|      3284.1|
|          EIRE|        50|          5|         1184|     2321.78|
|          EIRE|        51|          5|           95|      276.84|
|       Finland|        50|          1|         1254|       892.8|
|        France|        48|          4|         1299|     2808.16|
```

9: Git 6: TODO 4: Run Terminal Python Console Event Log

PEP 8: W391 blank line at end of file 37:5 CRLF UTF-8 4 spaces Python 3.7 (pyspark45) master

Aggregations

Country	WeekNumber	NumInvoices	TotalQuantity	InvoiceValue	RunningTotal
EIRE	48	7	2822	3147.23	3147.23
EIRE	49	5	1280	3284.1	6431.33
EIRE	50	5	1184	2321.78	8753.11
EIRE	51	5	95	276.84	9029.95
France	48	4	1299	2808.16	2808.16
France	49	9	2303	4527.01	7335.17
France	50	6	529	537.32	7872.49
France	51	5	847	1702.87	9575.36

LEARNING JOURNAL
www.learningjournal.guru

Windowing Aggregations?

udemy

Aggregations

Country	WeekNumber	NumInvoices	TotalQuantity	InvoiceValue	RunningTotal
EIRE	48	7	2822	3147.23	3147.23
EIRE	49	5	1280	3284.1	6431.33
EIRE	50	5	1184	2321.78	8753.11
EIRE	51	5	95	276.84	9029.95

France	48	4	1299	2808.16	2808.16
France	49	9	2303	4527.01	7335.17
France	50	6	529	537.32	7872.49
France	51	5	847	1702.87	9575.36

LEARNING JOURNAL
www.learningjournal.guru

Windowing Aggregate

1. Identify your partitioning columns.
2. Identify your ordering requirement.
3. Define your window start and end.

```python
if __name__ == "__main__":
    spark = SparkSession \
        .builder \
        .appName("Agg Demo") \
        .master("local[2]") \
        .getOrCreate()

    logger = Log4j(spark)

    summary_df = spark.read.parquet("data/summary.parquet")

    running_total_window = Window.partitionBy("Country") \
        .orderBy("WeekNumber") \
        .rowsBetween(Window.unboundedPreceding, Window.currentRow)

    summary_df.withColumn("RunningTotal",
                          f.sum("InvoiceValue").over(running_total_window)) \
        .show()
```

WindowingDemo.py

```python
        .master("local[2]") \
        .getOrCreate()

    logger = Log4j(spark)

    summary_df = spark.read.parquet("data/summary.parquet")

    running_total_window = Window.partitionBy("Country") \
        .orderBy("WeekNumber") \
        .rowsBetween(-2, Window.currentRow)

    summary_df.withColumn("RunningTotal",
                          f.sum("InvoiceValue").over(running_total_window)) \
        .show()

if __name__ == "__main__"
```